C Programming ESA Questions and Answers (Combined with Mapping)

Beginner Level

1. Categorize Identifiers (6.0 Marks) [Dec 2023, Page 1]

Question: Categorize the following as keywords, identifiers, or variables: `Return`, `me`, `short`, `Print`.

Answer:

- `Return`: Identifier (not a keyword, case-sensitive; `return` is a keyword).
- `me`: Identifier (valid variable name).
- `short`: Keyword (data type).
- `Print`: Identifier (not a keyword).

2. Types of Variables (6.0 Marks) [Jan–May 2024, Page 20]

Question: Identify the types of variables: `num`, `avg`, `count`, `ch`.

Answer:

- `num`: Likely `int` or `float` (context-dependent, e.g., for numbers).
- `avg`: Likely `float` or `double` (for averages).
- `count`: Likely `int` (for counting).
- `ch`: Likely `char` (for characters).

3. Valid/Invalid Variables (6.0 Marks) [Dec 2023, Page 2]

Question: Determine if the following are valid variable names: `num of digits`, `%avg`, `count_1`, `_total`.

Answer:

- `num of digits`: Invalid (contains spaces).
- `%avg`: Invalid (starts with %).
- `count_1`: Valid (letters, digits, underscore).
- `_total`: Valid (starts with underscore).

4. Explain Variables (4.0 Marks) [July 2023, Page 60]

Question: Explain the concept of variables in C with an example.

Answer:
A variable is a named memory location to store data. Example:

```c
#include <stdio.h>
int main() {
    int num = 10;
    printf("Value: %d\n", num);
    return 0;
}
```

Output: `Value: 10`

5. True or False Statements (5.0 Marks) [Jan–June 2024, Page 45]

Question: State true or false:

- `sizeof('\r')` is 2 bytes.
- C is a high-level language.
- `scanf` is case-sensitive.

Answer:

- `sizeof('\r')` is 2 bytes: False (1 byte for char in C).
- C is a high-level language: True (abstracts hardware details).
- `scanf` is case-sensitive: True (format specifiers like `%d` are case-sensitive).

6. Predict Output of Nested printf (4.0 Marks) [Dec 2023, Page 8]

Question: Predict the output:

```c
#include <stdio.h>
int main() {
    printf("%d", printf("Hello"));
    return 0;
}
```

Answer:

- **Explanation**: The inner `printf("Hello")` prints `Hello` and returns the number of characters printed (5). The outer `printf("%d", 5)` prints `5`.
- **Output**: `Hello5`

7. Check Even or Odd Using Ternary Operator (4.0 Marks) [Jan–May 2024, Page 30]

Question: Write a C program to check if a number is even or odd using the ternary operator.

Answer:

```c
#include <stdio.h>
int main() {
    int num;
```

```
    printf("Enter a number: ");
    scanf("%d", &num);
    printf("%s\n", num % 2 == 0 ? "Even" : "Odd");
    return 0;
}
```

8. Check Even or Odd Using Bitwise Operator (4.0 Marks) [July 2023, Page 61]

Question: Write a C program to check if a number is even or odd using a bitwise operator.

Answer:

- **Explanation**: The bitwise AND (`&`) with 1 checks the least significant bit (LSB). If LSB is 0, the number is even; if 1, it's odd.

```
#include <stdio.h>
int main() {
    int num;
    printf("Enter a number: ");
    scanf("%d", &num);
    printf("%s\n", (num & 1) == 0 ? "Even" : "Odd");
    return 0;
}
```

9. Sum of First and Last Digits (6.0 Marks) [Dec 2023, Page 10]

Question: Write a C program to find the sum of the first and last digits of a number.

Answer:

```
#include <stdio.h>
int main() {
    int num, first, last;
    printf("Enter a number: ");
    scanf("%d", &num);
    last = num % 10;
    while (num >= 10) num /= 10;
    first = num;
    printf("Sum: %d\n", first + last);
    return 0;
}
```

10. Check Middle Digit (6.0 Marks) [Jan–June 2024, Page 50]

Question: Write a C program to check if the middle digit of a three-digit number is even.

Answer:

- **Explanation**: For a three-digit number (e.g., 123), the middle digit is obtained by dividing by 10 (12) and taking modulo 10 (2).

```
#include <stdio.h>
int main() {
    int num;
    printf("Enter a three-digit number: ");
    scanf("%d", &num);
    int middle = (num / 10) % 10;
    printf("%s\n", middle % 2 == 0 ? "Even" : "Odd");
    return 0;
}
```

11. Sum of Natural Numbers Using Loop (5.0 Marks) [July 2023, Page 65]

Question: Write a C program to find the sum of first n natural numbers using a loop.

Answer:

```
#include <stdio.h>
int main() {
    int n, sum = 0;
    printf("Enter n: ");
    scanf("%d", &n);
    for (int i = 1; i <= n; i++) {
        sum += i;
    }
    printf("Sum: %d\n", sum);
    return 0;
}
```

12. Count Digits of a Number (7.0 Marks) [Jan–May 2024, Page 35]

Question: Write a C program to count the number of digits in a given number.

Answer:

- **Explanation**: Divide the number by 10 repeatedly until it becomes 0, counting each division. Handle the special case of 0 (1 digit).

```
#include <stdio.h>
int main() {
    int num, count = 0;
    printf("Enter a number: ");
    scanf("%d", &num);
    if (num == 0) count = 1;
    else {
        while (num != 0) {
            num /= 10;
```

```
            count++;
        }
    }
    printf("Digits: %d\n", count);
    return 0;
}
```

13. Types of Loops (2.0 Marks) [Dec 2023, Page 12]

Question: List the types of loops in C and explain their syntax.

Answer:

- `for` loop: `for (init; condition; update) { body; }`
- `while` loop: `while (condition) { body; }`
- `do-while` loop: `do { body; } while (condition);`

14. Predict Output of Loop (5.0 Marks) [Jan–June 2024, Page 55]

Question: Predict the output:

```
#include <stdio.h>
int main() {
    int i = 1;
    while (i <= 5) {
        printf("%d ", i);
        i++;
    }
    return 0;
}
```

Answer:

- **Explanation**: The loop runs from `i=1` to `i=5`, printing each value followed by a space. `i` increments after each print.
- **Output**: `1 2 3 4 5`

15. Convert for Loop to while Loop (6.0 Marks) [July 2023, Page 70]

Question: Convert the following `for` loop to a `while` loop:

```
for (int i = 1; i <= 5; i++) {
    printf("%d ", i);
}
```

Answer:

```
int i = 1;
while (i <= 5) {
    printf("%d ", i);
    i++;
}
```

16. Switch Construct (4.0 Marks) [Jan–May 2024, Page 40]

Question: Explain the `switch` construct in C with an example.

Answer:
The `switch` statement selects one of many code blocks based on a variable's value. Example:

```
#include <stdio.h>
int main() {
    int choice = 2;
    switch (choice) {
        case 1: printf("One\n"); break;
        case 2: printf("Two\n"); break;
        default: printf("Other\n");
    }
    return 0;
}
```

Output: `Two`

17. GCC Options (4.0 Marks) [Dec 2023, Page 15]

Question: Explain the following `gcc` options: `-o`, `-c`, `-E`.

Answer:

- `-o`: Specifies output file name (e.g., `gcc -o prog main.c`).
- `-c`: Compiles to object file without linking (e.g., `main.o`).
- `-E`: Preprocesses the code, outputs preprocessed source.

18. GCC Commands (6.0 Marks) [Jan–June 2024, Page 58]

Question: Write the `gcc` command to compile `main.c` to an executable named `program`.

Answer:

```
gcc -o program main.c
```

Intermediate Level

19. Program Development Life Cycle (6.0 Marks) [July 2023, Page 72]

Question: Explain the program development life cycle in C with a diagram.

Answer:
Stages: Problem Analysis, Design, Coding, Compilation, Debugging, Testing, Maintenance.
Diagram (text representation):

```
Problem Analysis -> Design -> Coding -> Compilation -> Debugging -> Testing -> Maintenance
```

20. Preprocessing Phase (6.0 Marks) [Jan–June 2024, Page 58]

Question: Explain the preprocessing phase in C compilation with examples.

Answer:
Preprocessing handles directives (`#include` , `#define`). Example:

```c
#define PI 3.14
#include <stdio.h>
int main() {
    printf("PI: %f\n", PI);
    return 0;
}
```

- **Explanation**: The preprocessor replaces `PI` with `3.14` and includes `stdio.h` before compilation.

21. String Functions (4.0 Marks) [Dec 2023, Page 18]

Question: Explain any four functions from `string.h` .

Answer:

- `strlen(str)` : Returns string length.
- `strcpy(dest, src)` : Copies src to dest.
- `strcmp(str1, str2)` : Compares strings (0 if equal).
- `strcat(dest, src)` : Concatenates src to dest.

22. Array of Integers (4.0 Marks) [Jan–May 2024, Page 45]

Question: Write a C program to read and display an array of 5 integers.

Answer:

```c
#include <stdio.h>
int main() {
    int arr[5];
    printf("Enter 5 integers: ");
    for (int i = 0; i < 5; i++) {
        scanf("%d", &arr[i]);
```

```
    }
    printf("Array: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    return 0;
}
```

23. Display Vowels (6.0 Marks) [Dec 2023, Page 21]

Question: Write a C function `disp_vowels` to display vowels in a string.

Answer:

```
#include <stdio.h>
void disp_vowels(char *str) {
    while (*str) {
        if (*str == 'a' || *str == 'e' || *str == 'i' || *str == 'o' || *str == 'u' ||
            *str == 'A' || *str == 'E' || *str == 'I' || *str == 'O' || *str == 'U') {
            printf("%c ", *str);
        }
        str++;
    }
    printf("\n");
}
int main() {
    char str[] = "Hello";
    disp_vowels(str);
    return 0;
}
```

Output: `e o`

24. Product of Array Elements (5.0 Marks) [Jan–June 2024, Page 60]

Question: Write a C program to find the product of elements in an array.

Answer:

```
#include <stdio.h>
int main() {
    int n, product = 1;
    printf("Enter array size: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter %d elements: ", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        product *= arr[i];
```

```
    }
    printf("Product: %d\n", product);
    return 0;
}
```

25. Sum of Array Elements (5.0 Marks) [July 2023, Page 75]

Question: Write a C program to find the sum of elements in an array.

Answer:

```
#include <stdio.h>
int main() {
    int n, sum = 0;
    printf("Enter array size: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter %d elements: ", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        sum += arr[i];
    }
    printf("Sum: %d\n", sum);
    return 0;
}
```

26. Read and Sum Array (6.0 Marks) [Jan–May 2024, Page 62]

Question: Write a C program to read an array and display the sum of its even and odd elements.

Answer:

```
#include <stdio.h>
int main() {
    int n, even_sum = 0, odd_sum = 0;
    printf("Enter array size: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter %d elements: ", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        if (arr[i] % 2 == 0) even_sum += arr[i];
        else odd_sum += arr[i];
    }
    printf("Even sum: %d\nOdd sum: %d\n", even_sum, odd_sum);
    return 0;
}
```

27. Find Pattern in String (7.0 Marks) [Jan–May 2024, Page 50]

Question: Write a C function to find the first occurrence of a pattern in a string.

Answer:

- **Explanation**: The function checks each substring of `str` of length equal to `pattern` for a match, returning the starting index or -1 if not found.

```
#include <stdio.h>
#include <string.h>
int find_pattern(char *str, char *pattern) {
    int n = strlen(str), m = strlen(pattern);
    for (int i = 0; i <= n - m; i++) {
        int j;
        for (j = 0; j < m; j++) {
            if (str[i + j] != pattern[j]) break;
        }
        if (j == m) return i;
    }
    return -1;
}
int main() {
    char str[] = "Hello World", pattern[] = "World";
    int pos = find_pattern(str, pattern);
    printf("Pattern found at index: %d\n", pos);
    return 0;
}
```

28. Structure Variable and Pointer (4.0 Marks) [Dec 2023, Page 25]

Question: Explain structure variable and structure pointer with an example.

Answer:

- Structure variable: Stores structure data.
- Structure pointer: Points to a structure. Example:

```
#include <stdio.h>
struct Student {
    int roll;
    char name[20];
};
int main() {
    struct Student s = {1, "Alice"};
    struct Student *p = &s;
    printf("Roll: %d, Name: %s\n", p->roll, p->name);
    return 0;
}
```

29. Characteristics of Structures (4.0 Marks) [July 2023, Page 80]

Question: List four characteristics of structures in C.

Answer:

- Groups different data types.
- User-defined data type.
- Accessed using dot (`.`) or arrow (`->`) operators.
- Can be passed to functions by value or address.

30. Structure Diagram (4.0 Marks) [Dec 2023, Page 26]

Question: Draw a diagram to represent a structure variable.

Answer:
Text representation:

```
struct Student {
    roll: [int]
    name: [char[20]]
}
s: | roll | name |
```

31. Dynamic Memory Functions (6.0 Marks) [Jan–June 2024, Page 65]

Question: Explain `malloc`, `calloc`, `free`, and `realloc` with examples.

Answer:

- `malloc` : Allocates memory.
- `calloc` : Allocates and initializes to zero.
- `free` : Deallocates memory.
- `realloc` : Resizes allocated memory. Example:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int *p = (int*)malloc(5 * sizeof(int));
    int *q = (int*)calloc(5, sizeof(int));
    p = (int*)realloc(p, 10 * sizeof(int));
    free(p);
    free(q);
    return 0;
}
```

32. Use of calloc and realloc (6.0 Marks) [Jan–May 2024, Page 55]

Question: Write a C program to demonstrate `calloc` and `realloc`.

Answer:

```c
#include <stdio.h>
#include <stdlib.h>
int main() {
    int *arr = (int*)calloc(3, sizeof(int));
    for (int i = 0; i < 3; i++) arr[i] = i + 1;
    printf("Original: ");
    for (int i = 0; i < 3; i++) printf("%d ", arr[i]);
    arr = (int*)realloc(arr, 5 * sizeof(int));
    for (int i = 3; i < 5; i++) arr[i] = i + 1;
    printf("\nResized: ");
    for (int i = 0; i < 5; i++) printf("%d ", arr[i]);
    free(arr);
    return 0;
}
```

33. Copy File with Comma (5.0 Marks) [Dec 2023, Page 30]

Question: Write a C program to copy a file, replacing spaces with commas.

Answer:

```c
#include <stdio.h>
int main() {
    FILE *in = fopen("input.txt", "r");
    FILE *out = fopen("output.txt", "w");
    if (!in || !out) {
        printf("File error\n");
        return 1;
    }
    char ch;
    while ((ch = fgetc(in)) != EOF) {
        fputc(ch == ' ' ? ',' : ch, out);
    }
    fclose(in);
    fclose(out);
    return 0;
}
```

34. Count Lines in CSV File (6.0 Marks) [Jan–May 2024, Page 60]

Question: Write a C program to count the number of lines in a CSV file.

Answer:

- **Explanation**: Increment a counter for each newline (`\n`). If the file doesn't end with a newline, count the last line.

```c
#include <stdio.h>
int main() {
    FILE *fp = fopen("data.csv", "r");
```

```
    if (!fp) {
        printf("File error\n");
        return 1;
    }
    int lines = 0;
    char ch;
    while ((ch = fgetc(fp)) != EOF) {
        if (ch == '\n') lines++;
    }
    if (ch != '\n') lines++;
    fclose(fp);
    printf("Lines: %d\n", lines);
    return 0;
}
```

35. File Length and Character Count (5.0 Marks) [July 2023, Page 85]

Question: Write a C program to find the length of a file and count specific characters.

Answer:

```
#include <stdio.h>
int main() {
    FILE *fp = fopen("input.txt", "r");
    if (!fp) {
        printf("File error\n");
        return 1;
    }
    fseek(fp, 0, SEEK_END);
    long length = ftell(fp);
    rewind(fp);
    int count = 0;
    char ch;
    while ((ch = fgetc(fp)) != EOF) {
        if (ch == 'a') count++;
    }
    fclose(fp);
    printf("Length: %ld\n'a' count: %d\n", length, count);
    return 0;
}
```

36. File Copy (5.0 Marks) [Dec 2023, Page 31]

Question: Write a C program to copy contents of one file to another.

Answer:

```
#include <stdio.h>
int main() {
    FILE *in = fopen("source.txt", "r");
```

```
    FILE *out = fopen("dest.txt", "w");
    if (!in || !out) {
        printf("File error\n");
        return 1;
    }
    char ch;
    while ((ch = fgetc(in)) != EOF) {
        fputc(ch, out);
    }
    fclose(in);
    fclose(out);
    return 0;
}
```

37. File Read and Display (6.0 Marks) [Jan–June 2024, Page 66]

Question: Write a C program to read and display the contents of a file.

Answer:

```
#include <stdio.h>
int main() {
    FILE *fp = fopen("input.txt", "r");
    if (!fp) {
        printf("File error\n");
        return 1;
    }
    char ch;
    while ((ch = fgetc(fp)) != EOF) {
        putchar(ch);
    }
    fclose(fp);
    return 0;
}
```

38. Selection Sort (6.0 Marks) [Jan–June 2024, Page 70]

Question: Write a C program to implement selection sort.

Answer:

- **Explanation**: Selection sort finds the minimum element in the unsorted portion and swaps it with the first unsorted element.

```
#include <stdio.h>
void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min_idx = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx]) min_idx = j;
```

```
            }
            int temp = arr[i];
            arr[i] = arr[min_idx];
            arr[min_idx] = temp;
        }
    }
    int main() {
        int arr[] = {5, 2, 8, 1, 9};
        int n = 5;
        selectionSort(arr, n);
        for (int i = 0; i < n; i++) printf("%d ", arr[i]);
        return 0;
    }
```

39. Bubble Sort (5.0 Marks) [Dec 2023, Page 35]

Question: Write a C program to implement bubble sort.

Answer:

- **Explanation**: Bubble sort repeatedly swaps adjacent elements if they are in the wrong order.

```
#include <stdio.h>
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
int main() {
    int arr[] = {5, 2, 8, 1, 9};
    int n = 5;
    bubbleSort(arr, n);
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    return 0;
}
```

40. Sorting Algorithm Characteristics (5.0 Marks) [Jan–May 2024, Page 71]

Question: List characteristics of selection sort and bubble sort.

Answer:

- **Selection Sort**: $O(n^2)$ time, in-place, unstable, finds min element per iteration.
- **Bubble Sort**: $O(n^2)$ time, in-place, stable, swaps adjacent elements.

41. Matrix Subtraction (4.0 Marks) [July 2023, Page 90]

Question: Write a C program to perform matrix subtraction.

Answer:

```
#include <stdio.h>
int main() {
    int m, n;
    printf("Enter rows and columns: ");
    scanf("%d %d", &m, &n);
    int a[m][n], b[m][n], c[m][n];
    printf("Enter matrix A:\n");
    for (int i = 0; i < m; i++) for (int j = 0; j < n; j++) scanf("%d", &a[i][j]);
    printf("Enter matrix B:\n");
    for (int i = 0; i < m; i++) for (int j = 0; j < n; j++) scanf("%d", &b[i][j]);
    for (int i = 0; i < m; i++) for (int j = 0; j < n; j++) c[i][j] = a[i][j] - b[i][j
    printf("Result:\n");
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) printf("%d ", c[i][j]);
        printf("\n");
    }
    return 0;
}
```

42. Sum of Diagonals (5.0 Marks) [Jan–May 2024, Page 65]

Question: Write a C program to find the sum of diagonals of a square matrix.

Answer:

- **Explanation**: Sum elements where row equals column (main diagonal) and row equals (n-1-column) (secondary diagonal). Subtract the middle element if n is odd (counted twice).

```
#include <stdio.h>
int main() {
    int n;
    printf("Enter size of matrix: ");
    scanf("%d", &n);
    int mat[n][n], sum = 0;
    printf("Enter matrix:\n");
    for (int i = 0; i < n; i++) for (int j = 0; j < n; j++) scanf("%d", &mat[i][j]);
    for (int i = 0; i < n; i++) sum += mat[i][i] + mat[i][n - 1 - i];
    if (n % 2 == 1) sum -= mat[n / 2][n / 2];
    printf("Sum of diagonals: %d\n", sum);
    return 0;
}
```

43. Matrix Transpose (5.0 Marks) [July 2023, Page 91]

Question: Write a C program to find the transpose of a matrix.

Answer:

```
#include <stdio.h>
int main() {
    int m, n;
    printf("Enter rows and columns: ");
    scanf("%d %d", &m, &n);
    int mat[m][n], trans[n][m];
    printf("Enter matrix:\n");
    for (int i = 0; i < m; i++) for (int j = 0; j < n; j++) scanf("%d", &mat[i][j]);
    for (int i = 0; i < m; i++) for (int j = 0; j < n; j++) trans[j][i] = mat[i][j];
    printf("Transpose:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) printf("%d ", trans[i][j]);
        printf("\n");
    }
    return 0;
}
```

44. Salesperson Calculations (4.0 Marks) [Dec 2023, Page 40]

Question: Write a C program to calculate total sales for 5 salespersons across 3 products.

Answer:

```
#include <stdio.h>
int main() {
    int sales[5][3];
    for (int i = 0; i < 5; i++) {
        printf("Enter sales for person %d (3 products): ", i + 1);
        for (int j = 0; j < 3; j++) scanf("%d", &sales[i][j]);
    }
    for (int i = 0; i < 5; i++) {
        int total = 0;
        for (int j = 0; j < 3; j++) total += sales[i][j];
        printf("Person %d total: %d\n", i + 1, total);
    }
    return 0;
}
```

45. Total Sales by Product (4.0 Marks) [Jan–June 2024, Page 75]

Question: Write a C program to calculate total sales for each product across 5 salespersons.

Answer:

```
#include <stdio.h>
int main() {
```

```
    int sales[5][3];
    for (int i = 0; i < 5; i++) {
        printf("Enter sales for person %d (3 products): ", i + 1);
        for (int j = 0; j < 3; j++) scanf("%d", &sales[i][j]);
    }
    for (int j = 0; j < 3; j++) {
        int total = 0;
        for (int i = 0; i < 5; i++) total += sales[i][j];
        printf("Product %d total: %d\n", j + 1, total);
    }
    return 0;
}
```

46. Swap Using Call by Reference (6.0 Marks) [Dec 2023, Page 41]

Question: Write a C function to swap two numbers using call by reference.

Answer:

- **Explanation**: Passing pointers allows the function to modify the original variables.

```
#include <stdio.h>
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
int main() {
    int x = 10, y = 20;
    printf("Before: x=%d, y=%d\n", x, y);
    swap(&x, &y);
    printf("After: x=%d, y=%d\n", x, y);
    return 0;
}
```

47. Function Terms (6.0 Marks) [Jan–May 2024, Page 76]

Question: Define formal parameters, actual parameters, call by value, and call by reference.

Answer:

- Formal parameters: Variables in function definition.
- Actual parameters: Values passed to function.
- Call by value: Copies values to function.
- Call by reference: Passes addresses to function.

48. Array Operations (4.0 Marks) [July 2023, Page 95]

Question: Write a C function to find the sum of even-indexed elements in an array.

Answer:

```
#include <stdio.h>
int sumEvenIndices(int arr[], int n) {
    int sum = 0;
    for (int i = 0; i < n; i += 2) sum += arr[i];
    return sum;
}
int main() {
    int arr[] = {1, 2, 3, 4, 5};
    printf("Sum: %d\n", sumEvenIndices(arr, 5));
    return 0;
}
```

49. Find Min and Max in Array (6.0 Marks) [Jan–May 2024, Page 70]

Question: Write a C function to find the minimum and maximum elements in an array.

Answer:

```
#include <stdio.h>
void findMinMax(int arr[], int n, int *min, int *max) {
    *min = *max = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] < *min) *min = arr[i];
        if (arr[i] > *max) *max = arr[i];
    }
}
int main() {
    int arr[] = {5, 2, 8, 1, 9};
    int min, max;
    findMinMax(arr, 5, &min, &max);
    printf("Min: %d, Max: %d\n", min, max);
    return 0;
}
```

50. Array Operations (5.0 Marks) [Dec 2023, Page 46]

Question: Write a C function to reverse an array.

Answer:

```
#include <stdio.h>
void reverseArray(int arr[], int n) {
    for (int i = 0; i < n / 2; i++) {
        int temp = arr[i];
        arr[i] = arr[n - 1 - i];
        arr[n - 1 - i] = temp;
    }
```

```
}
int main() {
    int arr[] = {1, 2, 3, 4, 5};
    reverseArray(arr, 5);
    for (int i = 0; i < 5; i++) printf("%d ", arr[i]);
    return 0;
}
```

51. my_strcat Function (6.0 Marks) [Jan–June 2024, Page 80]

Question: Write a C function `my_strcat` to concatenate two strings.

Answer:

```
#include <stdio.h>
char* my_strcat(char *dest, const char *src) {
    char *start = dest;
    while (*dest) dest++;
    while (*src) *dest++ = *src++;
    *dest = '\0';
    return start;
}
int main() {
    char dest[50] = "Hello", src[] = "World";
    my_strcat(dest, src);
    printf("%s\n", dest);
    return 0;
}
```

52. my_strcpy Function (4.0 Marks) [Dec 2023, Page 47]

Question: Write a C function `my_strcpy` to copy one string to another.

Answer:

```
#include <stdio.h>
char* my_strcpy(char *dest, const char *src) {
    char *start = dest;
    while (*src) *dest++ = *src++;
    *dest = '\0';
    return start;
}
int main() {
    char dest[50], src[] = "Hello";
    my_strcpy(dest, src);
    printf("%s\n", dest);
    return 0;
}
```

53. my_strlen Function (4.0 Marks) [Jan–May 2024, Page 77]

Question: Write a C function `my_strlen` to find the length of a string.

Answer:

```
#include <stdio.h>
int my_strlen(const char *str) {
    int len = 0;
    while (*str++) len++;
    return len;
}
int main() {
    char str[] = "Hello";
    printf("Length: %d\n", my_strlen(str));
    return 0;
}
```

54. Preprocessor Directives (5.0 Marks) [July 2023, Page 100]

Question: Explain `#include`, `#define`, `#ifdef`, `#endif` with examples.

Answer:

- `#include` : Includes header files.
- `#define` : Defines macros.
- `#ifdef`, `#endif` : Conditional compilation. Example:

```
#include <stdio.h>
#define MAX 10
#ifdef MAX
int main() {
    printf("MAX: %d\n", MAX);
    return 0;
}
#endif
```

55. Enumerations (5.0 Marks) [Jan–May 2024, Page 75]

Question: Explain enumerations in C with an example.

Answer:
Enums assign names to integral constants. Example:

```
#include <stdio.h>
enum Days {MON=1, TUE, WED};
int main() {
    enum Days day = TUE;
    printf("Day: %d\n", day);
```

```
    return 0;
}
```

Output: `Day: 2`

56. Storage Classes (4.0 Marks) [Dec 2023, Page 50]

Question: Explain storage classes in C.

Answer:

- `auto` : Local variables, default.
- `register` : Suggests register storage.
- `static` : Retains value between calls.
- `extern` : Global across files.

57. Volatile Keyword (4.0 Marks) [Jan–June 2024, Page 85]

Question: Explain the `volatile` keyword with an example.

Answer:
`volatile` prevents compiler optimization for variables that may change unexpectedly. Example:

```
#include <stdio.h>
volatile int flag = 0;
int main() {
    while (!flag) {
        printf("Waiting...\n");
    }
    return 0;
}
```

58. Extern Keyword (6.0 Marks) [Dec 2023, Page 51]

Question: Explain the `extern` keyword with an example.

Answer:
`extern` declares variables defined elsewhere. Example:

```
#include <stdio.h>
extern int global;
int global = 10;
int main() {
    printf("Global: %d\n", global);
    return 0;
}
```

Advanced Level

59. Return Last Node of Linked List (5.0 Marks) [Dec 2023, Page 55]

Question: Write a C function to return the last node of a linked list.

```
struct Node {
    int data;
    struct Node* next;
};
struct Node* getLastNode(struct Node* head);
```

Answer:

- **Explanation**: Traverse the list until `next` is NULL, returning the last node.

```
struct Node* getLastNode(struct Node* head) {
    if (!head) return NULL;
    while (head->next) head = head->next;
    return head;
}
```

60. Create Linked List (6.0 Marks) [Jan–May 2024, Page 80]

Question: Write a C function to create a linked list with n nodes.

```
struct Node* createList(int n);
```

Answer:

```
#include <stdlib.h>
struct Node* createList(int n) {
    struct Node *head = NULL, *temp = NULL;
    for (int i = 1; i <= n; i++) {
        struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
        newNode->data = i;
        newNode->next = NULL;
        if (!head) head = temp = newNode;
        else {
            temp->next = newNode;
            temp = newNode;
        }
    }
    return head;
}
```

61. Print Linked List (5.0 Marks) [Jan–June 2024, Page 90]

Question: Write a C function to print a linked list.

```
void printList(struct Node* head);
```

Answer:

```
#include <stdio.h>
void printList(struct Node* head) {
    while (head) {
        printf("%d ", head->data);
        head = head->next;
    }
    printf("\n");
}
```

62. Insert at Front of Linked List (6.0 Marks) [Dec 2023, Page 56]

Question: Write a C function to insert a node at the front of a linked list.

```
struct Node* insertFront(struct Node* head, int data);
```

Answer:

```
#include <stdlib.h>
struct Node* insertFront(struct Node* head, int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = head;
    return newNode;
}
```

63. Insert at End of Linked List (6.0 Marks) [Jan–May 2024, Page 81]

Question: Write a C function to insert a node at the end of a linked list.

```
struct Node* insertEnd(struct Node* head, int data);
```

Answer:

```
#include <stdlib.h>
struct Node* insertEnd(struct Node* head, int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
```

```
    if (!head) return newNode;
    struct Node* temp = head;
    while (temp->next) temp = temp->next;
    temp->next = newNode;
    return head;
}
```

64. Destroy Linked List (6.0 Marks) [Jan–June 2024, Page 91]

Question: Write a C function to destroy a linked list.

```
void destroyList(struct Node* head);
```

Answer:

```
#include <stdlib.h>
void destroyList(struct Node* head) {
    struct Node* temp;
    while (head) {
        temp = head;
        head = head->next;
        free(temp);
    }
}
```

65. Segregate Even and Odd Nodes (8.0 Marks) [Dec 2023, Page 57]

Question: Write a C function to segregate even and odd nodes in a linked list.

```
struct Node* segregateEvenOdd(struct Node* head);
```

Answer:

- **Explanation**: Maintain two lists (even and odd nodes), then link even list's end to odd list's start.

```
#include <stdlib.h>
struct Node* segregateEvenOdd(struct Node* head) {
    struct Node *evenStart = NULL, *evenEnd = NULL, *oddStart = NULL, *oddEnd = NULL;
    struct Node* curr = head;
    while (curr) {
        int val = curr->data;
        if (val % 2 == 0) {
            if (!evenStart) evenStart = evenEnd = curr;
            else {
                evenEnd->next = curr;
                evenEnd = curr;
            }
        } else {
```

```
            if (!oddStart) oddStart = oddEnd = curr;
            else {
                oddEnd->next = curr;
                oddEnd = curr;
            }
        }
        curr = curr->next;
    }
    if (!evenStart || !oddStart) return head;
    evenEnd->next = oddStart;
    oddEnd->next = NULL;
    return evenStart;
}
```

66. Iterative Binary Search (5.0 Marks) [July 2023, Page 105]

Question: Write a C function for iterative binary search.

```
int binarySearch(int arr[], int n, int key);
```

Answer:

- **Explanation**: Halve the search space iteratively, comparing the middle element to the key.

```
int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == key) return mid;
        if (arr[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
```

67. Recursive Binary Search (6.0 Marks) [Jan–May 2024, Page 85]

Question: Write a C function for recursive binary search.

```
int binarySearchRecursive(int arr[], int low, int high, int key);
```

Answer:

- **Explanation**: Recursively halve the search space, base case is when `low > high`.

```
int binarySearchRecursive(int arr[], int low, int high, int key) {
    if (low > high) return -1;
    int mid = low + (high - low) / 2;
```

```
    if (arr[mid] == key) return mid;
    if (arr[mid] < key) return binarySearchRecursive(arr, mid + 1, high, key);
    return binarySearchRecursive(arr, low, mid - 1, key);
}
```

68. Recursive Sequence (4.0 Marks) [Dec 2023, Page 60]

Question: Write a recursive C function to calculate the sum of the first n terms of the sequence: 1, 3, 6, 10, ...

Answer:

- **Explanation**: The nth term is `n*(n+1)/2`. Sum recursively by adding nth term to sum of (n-1) terms.

```
int sequenceSum(int n) {
    if (n == 0) return 0;
    return (n * (n + 1)) / 2 + sequenceSum(n - 1);
}
```

69. Recursion Definition (5.0 Marks) [Jan–June 2024, Page 95]

Question: Define recursion and write a recursive function to find the sum of digits of a number.

Answer:
Recursion: Function calling itself. Example:

```
int sumDigits(int n) {
    if (n == 0) return 0;
    return (n % 10) + sumDigits(n / 10);
}
```

- **Explanation**: Extract the last digit (`n % 10`) and recursively sum the rest (`n / 10`).

70. Error Handling (4.0 Marks) [Jan–May 2024, Page 90]

Question: Explain `errno` and `strerror` in C with an example.

Answer:
`errno`: Error code set by system calls.
`strerror`: Returns error message for `errno`. Example:

```
#include <stdio.h>
#include <errno.h>
#include <string.h>
int main() {
    FILE *fp = fopen("nonexistent.txt", "r");
    if (!fp) printf("Error: %s\n", strerror(errno));
    return 0;
}
```

71. Callback Functions (5.0 Marks) [July 2023, Page 110]

Question: Explain callback functions in C and provide an example.

Answer:
Callback: Function passed as an argument. Example:

```
#include <stdio.h>
void process(int a, int b, int (*callback)(int, int)) {
    printf("Result: %d\n", callback(a, b));
}
int add(int a, int b) { return a + b; }
int main() {
    process(5, 3, add);
    return 0;
}
```

Output: `Result: 8`

72. Conditional Compilation (4.0 Marks) [Dec 2023, Page 65]

Question: Explain conditional compilation in C.

Answer:
Uses `#if`, `#ifdef`, `#else`, `#endif` to include/exclude blocks. Example:

```
#include <stdio.h>
#ifdef DEBUG
    printf("Debug mode\n");
#endif
int main() {
    printf("Running\n");
    return 0;
}
```

73. Macro with String (5.0 Marks) [Jan–June 2024, Page 100]

Question: Predict the output:

```
#include <stdio.h>
#define STR(x) #x
int main() {
    printf("%s\n", STR(Hello));
    return 0;
}
```

Answer:

- **Explanation**: The `#x` operator converts the macro argument `Hello` to a string literal `"Hello"`.
- **Output**: `Hello`

74. Macro with Expression (5.0 Marks) [Jan–May 2024, Page 96]

Question: Predict the output:

```
#include <stdio.h>
#define SQUARE(x) x * x
int main() {
    printf("%d\n", SQUARE(3 + 2));
    return 0;
}
```

Answer:

- **Explanation**: Expands to `3 + 2 * 3 + 2`. Due to precedence, `2 * 3 = 6`, then `3 + 6 + 2 = 11`. Parentheses in macro definition (e.g., `(x) * (x)`) would give `(3 + 2) * (3 + 2) = 25`.
- **Output**: `11`

75. Macro Output (6.0 Marks) [Dec 2023, Page 66]

Question: Predict the output:

```
#include <stdio.h>
#define MERGE(x, y) x##y
int main() {
    int xy = 10;
    printf("%d\n", MERGE(x, y));
    return 0;
}
```

Answer:

- **Explanation**: The `##` operator concatenates `x` and `y` to form `xy`, referencing the variable `xy`.
- **Output**: `10`

76. Structure vs. Union (4.0 Marks) [July 2023, Page 115]

Question: Differentiate between structure and union in C.

Answer:

- **Structure**: Allocates memory for all members.
- **Union**: Allocates memory for the largest member; members share memory.

77. Bit Fields and Enums (5.0 Marks) [Jan–May 2024, Page 95]

Question: Explain bit fields and enums with examples.

Answer:

- Bit fields: Specify bits for structure members.
- Enums: Named integer constants. Example:

```
#include <stdio.h>
struct Bits {
    unsigned int a:2;
};
enum Days {MON=1, TUE};
int main() {
    struct Bits b = {2};
    printf("%d\n", b.a);
    return 0;
}
```

78. Structure Operations (5.0 Marks) [Dec 2023, Page 70]

Question: Write a C program to read and display a structure using a pointer.

Answer:

```
#include <stdio.h>
struct Student {
    int roll;
    char name[20];
};
int main() {
    struct Student s, *p = &s;
    printf("Enter roll and name: ");
    scanf("%d %s", &p->roll, p->name);
    printf("Roll: %d, Name: %s\n", p->roll, p->name);
    return 0;
}
```

79. Dynamic Memory for Structure (6.0 Marks) [Jan–June 2024, Page 105]

Question: Write a C program to allocate memory dynamically for a structure.

Answer:

```
#include <stdio.h>
#include <stdlib.h>
struct Student {
    int roll;
    char name[20];
};
int main() {
    struct Student* s = (struct Student*)malloc(sizeof(struct Student));
```

```
    printf("Enter roll and name: ");
    scanf("%d %s", &s->roll, s->name);
    printf("Roll: %d\n, Name: %s\n", s->roll, s->name);
    free(s);
    return 0;
}
```

80. Array of Pointers (5.0 Marks) [Jan–May 2024, Page 97]

Question: Write a C program to demonstrate an array of pointers to integers.

Answer:

```
#include <stdio.h>
int main() {
    int a = 1, b = 2, c = 3;
    int *arr[3] = {&a, &b, &c};
    for (int i = 0; i < 3; i++) {
        printf("%d ", *arr[i]);
    }
    printf("\n");
    return 0;
}
```

81. Array of Structures (5.0 Marks) [Dec 2023, Page 71]

Question: Write a C program to demonstrate an array of structures.

Answer:

```
#include <stdio.h>
struct Student {
    int roll;
    char name[20];
};
int main() {
    struct Student s[3];
    for (int i = 0; i < 3; i++) {
        printf("Enter roll %d and name: ", i + 1);
        scanf("%d %s", &s[i].roll, s[i].name);
    }
    for (int i = 0; i < 3; i++) {
        printf("Roll: %d, Name: %s\n", s[i].roll, s[i].name);
    }
    return 0;
}
```

82. Employee Attendance Structure (6.0 Marks) [Jan–May 2024, Page 100]

Question: Write a C program to manage employee attendance using a structure with alias, array, and pointer.

Answer:

```
#include <stdio.h>
typedef struct {
    int id;
    char name[50];
    int days;
} Employee;
int main() {
    Employee emp[3];
    Employee *p = emp;
    for (int i = 0; i < 3; i++) {
        printf("Enter ID %d, name, days: ", i + 1);
        scanf("%d %s %d", &p[i].id, p[i].name, &p[i].days);
    }
    int total = 0;
    for (int i = 0; i < 3; i++) {
        total += p[i].days;
        printf("ID: %d, Name: %s, Days: %d\n", p[i].id, p[i].name, p[i].days);
    }
    printf("Total days: %d\n", total);
    return 0;
}
```

83. Complex Expression Output (7.0 Marks) [Dec 2023, Page 75]

Question: Predict the output:

```
#include <stdio.h>
int main() {
    int a = 5, b = 10, c;
    c = a++ + ++b + ++a + b++;
    printf("%d %d %d\n", a, b, c);
    return 0;
}
```

Answer:

- **Explanation**: Evaluate left to right:
  - `a++` uses 5, then `a=6`.
  - `++b` makes `b=11`, uses 11.
  - `++a` makes `a=7`, uses 7.
  - `b++` uses 11, then `b=12`.
  - `c = 5 + 11 + 7 + 11 = 34`.
  - Final: `a=7`, `b=12`, `c=34`.
- **Output**: `7 12 34`

84. String Tokenization Output (6.0 Marks) [Jan–June 2024, Page 110]

Question: Predict the output:

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[] = "Hello,World";
    char *token = strtok(str, ",");
    while (token != NULL) {
        printf("%s\n", token);
        token = strtok(NULL, ",");
    }
    return 0;
}
```

Answer:

- **Explanation**: `strtok` splits `str` at commas. First call returns `"Hello"`, second call (with `NULL`) returns `"World"`.
- **Output**:

```
Hello
World
```

85. Enum Output (5.0 Marks) [July 2023, Page 120]

Question: Predict the output:

```
#include <stdio.h>
enum Color {RED, GREEN, BLUE};
int main() {
    enum Color c = GREEN;
    printf("%d\n", c);
    return 0;
}
```

Answer:

- **Explanation**: Enums assign integers starting from 0. `GREEN` is 1 (after `RED=0`).
- **Output**: `1`

86. Macro Output (4.0 Marks) [Jan–May 2024, Page 98]

Question: Predict the output:

```
#include <stdio.h>
#define DOUBLE(x) 2 * x
int main() {
```

```
    printf("%d\n", DOUBLE(3));
    return 0;
}
```

Answer:

- **Explanation**: Expands to `2 * 3 = 6`. Note: If used as `DOUBLE(3 + 2)`, it would expand to `2 * 3 + 2 = 8`, not 10.
- **Output**: `6`

87. Structure Output (4.0 Marks) [Dec 2023, Page 76]

Question: Predict the output:

```
#include <stdio.h>
struct Point {
    int x, y;
};
int main() {
    struct Point p = {10, 20};
    printf("%d\n", p.x + p.y);
    return 0;
}
```

Answer:

- **Explanation**: Structure `p` has `x=10`, `y=20`. Print their sum: `10 + 20 = 30`.
- **Output**: `30`

88. String Output (5.0 Marks) [Jan–June 2024, Page 111]

Question: Predict the output:

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[] = "Hello";
    printf("%d\n", strlen(str));
    return 0;
}
```

Answer:

- **Explanation**: `strlen` returns the length of the string excluding the null terminator.
- **Output**: `5`

89. Array Output (5.0 Marks) [July 2023, Page 121]

Question: Predict the output:

```c
#include <stdio.h>
int main() {
    int arr[] = {1, 2, 3};
    printf("%d\n", arr[1]);
    return 0;
}
```

Answer:

- **Explanation**: `arr[1]` accesses the second element (index 1).
- **Output**: `2`

90. Pointer Output (5.0 Marks) [Dec 2023, Page 77]

Question: Predict the output:

```c
#include <stdio.h>
int main() {
    int a = 5;
    int *p = &a;
    printf("%d\n", *p);
    return 0;
}
```

Answer:

- **Explanation**: `p` points to `a`. Dereferencing `*p` gives `a`'s value, 5.
- **Output**: `5`

91. Preprocessor Output (6.0 Marks) [Jan–May 2024, Page 99]

Question: Predict the output:

```c
#include <stdio.h>
#define CALC(x) x*x
int main() {
    int a = 4;
    printf("%d\n", CALC(a++));
    return 0;
}
```

Answer:

- **Explanation**: Expands to `a++ * a++`. Undefined behavior due to multiple modifications of `a` without sequence point. Possible result: `a=4*5=20`, `a=6`. Behavior may vary.
- **Output**: `20` (compiler-dependent)

92. Static Variable Output (6.0 Marks) [Jan–June 2024, Page 112]

Question: Predict the output:

```
#include <stdio.h>
void func() {
    static int x = 0;
    x++;
    printf("%d ", x);
}
int main() {
    func();
    func();
    return 0;
}
```

Answer:

- **Explanation**: `static` variable `x` retains its value between calls. First call: `x=1` . Second call: `x=2` .
- **Output**: `1 2`

93. Pointer Arithmetic Output (6.0 Marks) [Dec 2023, Page 78]

Question: Predict the output:

```
#include <stdio.h>
int main() {
    int arr[] = {1, 2, 3};
    int *p = arr;
    printf("%d ", *(p + 1));
    return 0;
}
```

Answer:

- **Explanation**: `p` points to `arr[0]` . `p + 1` points to `arr[1]` . Dereferencing gives `arr[1] = 2` .
- **Output**: `2`

94. Function Pointer Output (6.0 Marks) [Jan–May 2024, Page 101]

Question: Predict the output:

```
#include <stdio.h>
int add(int a, int b) { return a + b; }
int main() {
    int (*fp)(int, int) = add;
    printf("%d\n", fp(2, 3));
    return 0;
}
```

Answer:

- **Explanation**: `fp` is a function pointer to `add`. Calling `fp(2, 3)` invokes `add(2, 3) = 5`.
- **Output**: `5`

95. File Operation Output (6.0 Marks) [July 2023, Page 122]

Question: Predict the output:

```
#include <stdio.h>
int main() {
    FILE *fp = fopen("test.txt", "w");
    fprintf(fp, "Hello");
    fclose(fp);
    fp = fopen("test.txt", "r");
    char ch;
    while ((ch = fgetc(fp)) != EOF) {
        printf("%c", ch);
    }
    fclose(fp);
    return 0;
}
```

Answer:

- **Explanation**: Writes `"Hello"` to `test.txt`, then reads and prints each character.
- **Output**: `Hello`

96. Linked List Output (6.0 Marks) [Dec 2023, Page 79]

Question: Predict the output:

```
#include <stdio.h>
struct Node {
    int data;
    struct Node* next;
};
int main() {
    struct Node n1 = {10, NULL};
    struct Node n2 = {20, NULL};
    n1.next = &n2;
    printf("%d\n", n1.next->data);
    return 0;
}
```

Answer:

- **Explanation**: `n1.next` points to `n2`. Accessing `n1.next->data` gives `n2.data = 20`.
- **Output**: `20`

97. Recursive Digit Sum Output (6.0 Marks) [Jan–June 2024, Page 113]

Question: Predict the output:

```
#include <stdio.h>
int sumDigits(int n) {
    if (n == 0) return 0;
    return (n % 10) + sumDigits(n / 10);
}
int main() {
    printf("%d\n", sumDigits(123));
    return 0;
}
```

Answer:

- **Explanation**: For 123: `3 + sumDigits(12) = 3 + (2 + sumDigits(1)) = 3 + 2 + (1 + sumDigits(0)) = 3 + 2 + 1 = 6`.
- **Output**: `6`

98. Array Pointer Output (6.0 Marks) [Jan–May 2024, Page 102]

Question: Predict the output:

```
#include <stdio.h>
int main() {
    int arr[] = {1, 2, 3};
    int (*p)[3] = &arr;
    printf("%d\n", (*p)[1]);
    return 0;
}
```

Answer:

- **Explanation**: `p` is a pointer to an array of 3 ints. `(*p)[1]` dereferences to `arr[1] = 2`.
- **Output**: `2`

99. Dynamic Memory Output (6.0 Marks) [Dec 2023, Page 80]

Question: Predict the output:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int *p = (int*)malloc(2 * sizeof(int));
    p[0] = 1;
    p[1] = 2;
    printf("%d %d\n", p[0], p[1]);
    free(p);
    return 0;
}
```

Answer:

- **Explanation**: Allocates memory for 2 integers, assigns values, and prints them.
- **Output**: `1 2`

100. String Reverse Output (6.0 Marks) [Jan–May 2024, Page 30]

Question: Predict the output:

```
#include <stdio.h>
#include <string.h>
void reverse(char *str) {
    int i, j;
    char temp;
    for (i = 0, j = strlen(str) - 1; i < j; i++, j--) {
        temp = str[i];
        str[i] = str[j];
        str[j] = temp;
    }
}
int main() {
    char str[] = "Hello";
    reverse(str);
    printf("%s\n", str);
    return 0;
}
```

Answer:

- **Explanation**: The `reverse` function swaps characters from start and end, moving inward. `"Hello"` becomes `"olleH"`.
- **Output**: `olleH`

101. Count Occurrences in String (6.0 Marks) [Dec 2023, Page 18]

Question: Write a C function to count the occurrences of a character in a string.

```
int count_char(char *str, char ch);
```

Answer:

```
int count_char(char *str, char ch) {
    int count = 0;
    while (*str) {
        if (*str == ch) count++;
        str++;
    }
```

```
    return count;
}
```

102. Linked List Length (6.0 Marks) [Jan–June 2024, Page 90]

Question: Write a C function to find the number of nodes in a linked list.

```
struct Node {
    int data;
    struct Node* next;
};
int getLength(struct Node* head);
```

Answer:

- **Explanation**: Traverse the list, counting nodes until `head` is NULL.

```
int getLength(struct Node* head) {
    int count = 0;
    while (head) {
        count++;
        head = head->next;
    }
    return count;
}
```

103. Delete Node in Linked List (6.0 Marks) [Dec 2023, Page 55]

Question: Write a C function to delete a node with a given key from a linked list.

```
struct Node {
    int data;
    struct Node* next;
};
struct Node* deleteNode(struct Node* head, int key);
```

Answer:

- **Explanation**: If the key is in the head, update `head`. Otherwise, traverse to find the key, link the previous node to the next, and free the node.

```
#include <stdlib.h>
struct Node* deleteNode(struct Node* head, int key) {
    struct Node *current = head, *prev = NULL;
    if (current && current->data == key) {
        head = current->next;
        free(current);
        return head;
```

```
    }
    while (current && current->data != key) {
        prev = current;
        current = current->next;
    }
    if (current == NULL) return head;
    prev->next = current->next;
    free(current);
    return head;
}
```

104. Recursive Sum of Array (6.0 Marks) [Jan–May 2024, Page 85]

Question: Write a recursive C function to find the sum of elements in an integer array.

```
int arraySum(int arr[], int n);
```

Answer:

- **Explanation**: Base case: empty array returns 0. Recursive case: sum last element and rest of array.

```
int arraySum(int arr[], int n) {
    if (n <= 0) return 0;
    return arr[n - 1] + arraySum(arr, n - 1);
}
```

105. File Read and Count Words (6.0 Marks) [Jan–June 2024, Page 95]

Question: Write a C program to read a text file "input.txt" and count the number of words.

Answer:

- **Explanation**: Track word boundaries by detecting transitions from non-space to space characters.

```
#include <stdio.h>
int main() {
    FILE *fp = fopen("input.txt", "r");
    if (fp == NULL) {
        printf("File error\n");
        return 1;
    }
    int count = 0;
    char ch;
    int in_word = 0;
    while ((ch = fgetc(fp)) != EOF) {
        if (ch == ' ' || ch == '\n' || ch == '\t') {
            in_word = 0;
        } else if (in_word == 0) {
            in_word = 1;
```

```
            count++;
        }
    }
    fclose(fp);
    printf("Number of words: %d\n", count);
    return 0;
}
```

106. Command Line Arguments Sum (5.0 Marks) [Dec 2023, Page 40]

Question: Write a C program to compute the sum of integer arguments passed via command line.

Answer:

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[]) {
    int sum = 0;
    for (int i = 1; i < argc; i++) {
        sum += atoi(argv[i]);
    }
    printf("Sum: %d\n", sum);
    return 0;
}
```

107. Bit Manipulation Toggle (5.0 Marks) [Jan–May 2024, Page 90]

Question: Write a C function to toggle the nth bit of an integer.

```
int toggleBit(int num, int n);
```

Answer:

- **Explanation**: Use XOR (`^`) with `1 << n` to toggle the nth bit (0 to 1 or 1 to 0).

```
int toggleBit(int num, int n) {
    return num ^ (1 << n);
}
```

108. Dynamic Array of Structures (6.0 Marks) [Jan–June 2024, Page 100]

Question: Write a C program to create a dynamic array of `Student` structures and read/display their data.

```
struct Student {
    int roll_no;
    char name[50];
};
```

Answer:

- **Explanation**: Dynamically allocate memory for `n Student` structures using `malloc`. Use `fgets` for safe string input, removing the trailing newline. Free memory after use to prevent leaks.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
struct Student {
int roll_no;
char name[50];
};
int main() {
int n;
printf("Enter number of students: ");
scanf("%d", &n);
struct Student *students = (struct Student*)malloc(n * sizeof(struct Student));
if (students == NULL) {
printf("Memory allocation failed\n");
return 1;
}
for (int i = 0; i < n; i++) {
printf("Enter roll no and name for student %d: ", i + 1);
scanf("%d", &students[i].roll_no);
getchar(); // Clear newline
fgets(students[i].name, 50, stdin);
students[i].name[strcspn(students[i].name, "\n")] = '\0'; // Remove trailing newline
}
printf("\nStudent Details:\n");
for (int i = 0; i < n; i++) {
printf("Student %d: Roll No: %d, Name: %s\n", i + 1, students[i].roll_no, students[i].name);
}
free(students);
return 0;
}
```

109. Bit Manipulation Count Set Bits (5.0 Marks) [Jan–May 2024, Page 91]

Question: Write a C function to count the number of set bits (1s) in an integer.

```
int countSetBits(int num);
```

Answer:

- **Explanation**: Use `num & 1` to check the least significant bit and right-shift `num` to process each bit. Alternatively, `num & (num - 1)` clears the rightmost set bit, counting iterations until `num` is 0 (Brian Kernighan's algorithm is shown here for efficiency).

```
#include <stdio.h>
int countSetBits(int num){
int count = 0;
while (num){
num &= (num - 1); // Clear rightmost set bit
count++;
}
return count;
}
int main(){
int num;
printf("Enter a number: ");
scanf("%d", &num);
printf("Number of set bits: %d\n", countSetBits(num));
return 0;
}
```

110. Predict Pointer Output (6.0 Marks) [Dec 2023, Page 81]

Question: Predict the output:

```
#include <stdio.h>
int main() {
    int a = 10, b = 20;
    int *p = &a, *q = &b;
    *p = *q;
    printf("%d %d\n", a, b);
    return 0;
}
```

Answer:

- **Explanation**: `p` points to `a`, and `q` points to `b`. The assignment `*p = *q` copies the value pointed to by `q` (20) to the location pointed to by `p` (i.e., `a`). Thus, `a` becomes 20, while `b` remains unchanged.
- **Output**: `20 20`